

Member 1 — Database Foundation

Task	What we did	Status
1. Database creation	Created the <code>epms</code> MySQL database	☒
2. Core tables	Created the main EPMS tables required by the system	☒
3. User tables	Created <code>users</code> , <code>roles</code> , and user-role structure	☒
4. Authentication-related tables	Added supporting tables such as sessions/password-reset related structures	☒
5. EPMS module tables	Created tables supporting manuscript, editorial, production, inventory, orders, payment, royalty, etc.	☒
6. Primary/foreign keys	Established relationships between entities	☒
7. Constraints	Added <code>NOT NULL</code> , <code>UNIQUE</code> , <code>enum</code> /relationship constraints where required	☒
8. Timestamps	Added <code>created_at</code> / <code>updated_at</code> handling	☒
9. Database verification	Compared the database structure against the Java authentication requirements	☒
10. Integration checking	Identified the <code>users</code> schema mismatch that caused the registration errors	☒

Member 1 overall

Database foundation completed.

The remaining consideration is keeping the final SQL schema and Java entities aligned during full integration.

Member 3 — Authentication Foundation

Task	What we did	Status
1. User Entity	Created the <code>User</code> JPA entity	☒
2. Role handling	Added role support for <code>ADMIN</code> , <code>AUTHOR</code> , <code>EDITOR</code> , <code>REVIEWER</code>	☒
3. User Repository	Created repository methods such as finding users by email	☒
4. Register DTO	Created <code>RegisterRequest</code>	☒
5. Login DTO	Created <code>LoginRequest</code>	☒
6. API response structure	Created reusable <code>ApiResponse</code>	☒
7. Validation	Added request validation	☒
8. Exception handling	Added global exception handling	☒

Task	What we did	Status
9. Password encryption	Implemented BCrypt password hashing	☒
10. Registration logic	Implemented user registration	☒
11. Duplicate email checking	Prevented registration using an existing email	☒
12. Login logic	Implemented email/password authentication	☒
13. JWT generation	Generated JWT after successful login	☒
14. Login response	Returned token + user information	☒
15. Authentication Controller	Created <code>/api/auth/register</code> and <code>/api/auth/login</code> endpoints	☒
16. Registration testing	Tested successfully through Swagger — 200 OK	☒
17. Login testing	Tested successfully through Swagger — 200 OK and JWT returned	☒

Member 3 overall

☒ 100% complete and tested.

Member 4 — Security Foundation

Task	What we did	Status
1. PasswordEncoder	Configured BCrypt password encoder	☒
2. SecurityConfig	Configured Spring Security	☒
3. Stateless sessions	Configured <code>SessionCreationPolicy.STATELESS</code>	☒
4. Public authentication routes	Allowed <code>/api/auth/**</code> without authentication	☒
5. Swagger access	Allowed Swagger/OpenAPI endpoints without authentication	☒
6. Protected routes	Configured all other routes to require authentication	☒
7. Form login disabled	Disabled default form-based authentication	☒
8. HTTP Basic disabled	Disabled HTTP Basic authentication	☒
9. CustomUserDetailsService	Connected Spring Security to the EPMS users	☒
10. JWT Service	Created JWT generation and validation logic	☒
11. JWT secret configuration	Added JWT secret and expiration to <code>application.properties</code>	☒
12. JWT Authentication Filter	Read <code>Authorization: Bearer <token></code> from requests	☒

Task	What we did	Status
13. JWT validation	Validated token signature, username and expiration	☑
14. SecurityContext	Added authenticated user to Spring Security context	☑
15. Swagger Bearer authentication	Configured Swagger's <code>bearerAuth</code>	☑
16. Protected endpoint	Created <code>/api/security-test/protected</code>	☑
17. Unauthorized test	Called protected endpoint without JWT → 403	☑
18. Authorized test	Called protected endpoint with valid JWT → 200 OK	☑
19. End-to-end security test	Verified Login → JWT → Swagger → Filter → Protected endpoint	☑

Member 4 overall

☑ 100% complete and tested.

The actual workflow we completed

This is probably the **most useful short version** for your task board:

Member 1

Database → Tables → Relationships → Constraints → Authentication tables → EPMS module tables → Database verification

Member 3

User Entity → Repository → DTOs → Validation → BCrypt → Registration → Login → JWT generation → Authentication Controller → Swagger testing

Member 4

PasswordEncoder → SecurityConfig → CustomUserDetailsService → JWT Service → JWT Filter → Swagger Bearer Auth → Protected Endpoint → Unauthorized Test → Authorized JWT Test

And the final security test proved:

```
Register
  ↓
Login
  ↓
```

JWT generated
↓
Swagger Authorize
↓
Bearer JWT
↓
JWT Filter
↓
Spring Security
↓
Protected Endpoint
↓
200 OK ☒

So these are the **core foundation tasks you and the relevant members actually completed**, rather than a generic list of Spring Boot tasks.